Enhancing Building Energy Efficiency with IoT-Driven Hybrid Deep Learning Models for Accurate Energy Consumption Prediction

IoT Networks Project Phases 2 & 3 Presentation

by Mahan Bayhaghi

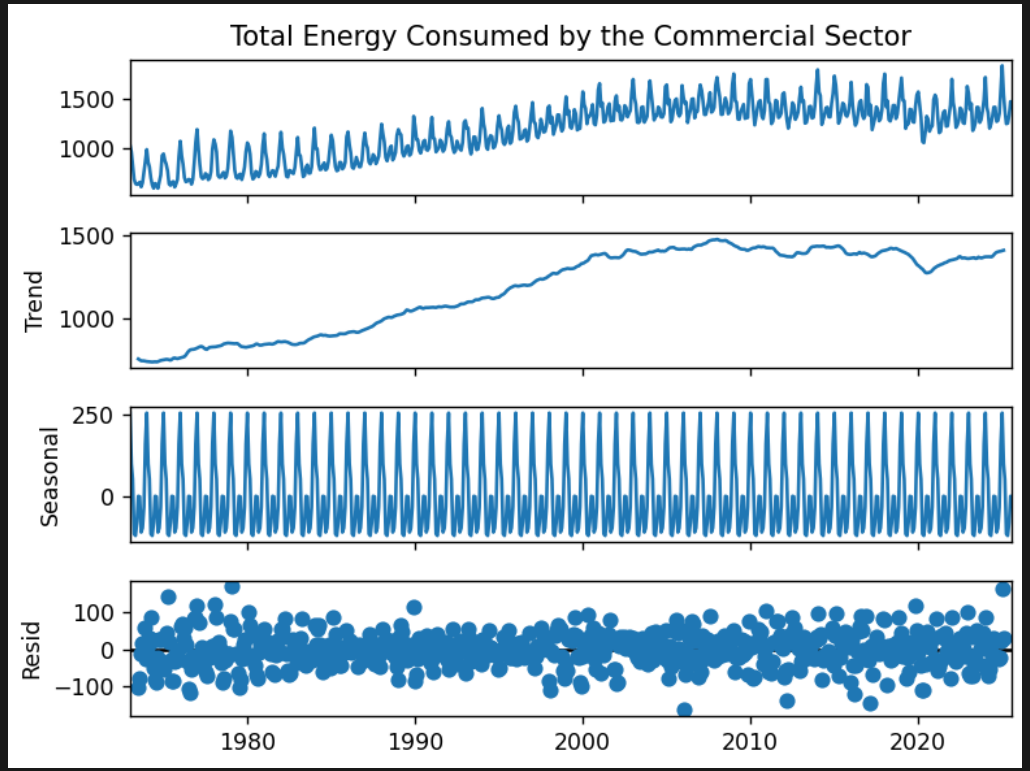


Sharif University of Technology
Dept. of Computer Engineering
Fall 2025

Dataset and Preprocessing

- **Data Source:** Utilized publicly available energy data from the EIA (Energy Information Administration).
- **Time Series Formatting:** Data structured into sequences with a defined look-back window.
- **MinMax Scaling:** used to standardize the input features between 0 and 1.
- **Why MinMax Scaling?**
    - Neural networks, especially LSTMs, converge faster and perform more consistently when input data is normalized, preventing larger values from dominating the learning process.

Dataset EDA


Total Energy Consumed by the Commercial Sector
1500
1000
Trend
1500
1000
Seasonal
250
0
Resid
100
0
−100
1980
1990
2000
2010
2020

Paper’s Claims

Model Name	Prediction Performance
CNN	Weak. Fails to capture temporal dependencies
BiLSTM	Better than CNN yet have lags (delays)
CNN-BiLSTM (Proposed)	**Proposed method** **Better than both standalone CNN and BiLSTM** **Captures both temporal and spatial dependencies**
Ensemble Models (LGBM, XGBoost, CatBoost)	Inherently great for prediction Capturing more meaningful features (lags and means) Yet fails against proposed model

Performance Metrics

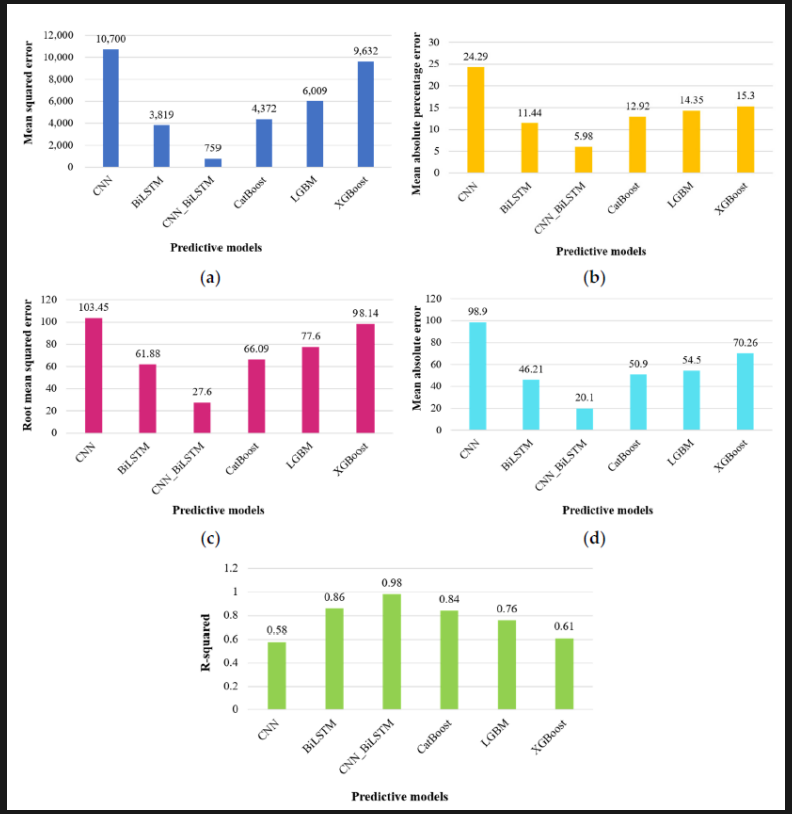
1. **MSE / RMSE:** Sensitive to large errors due to squaring, ideal for identifying significant deviations from actual values.
2. **MAE:** Represents the average absolute error, offering a linear score that is intuitive for estimating operational costs and typical error magnitudes.
3. **R-squared (R^2):** Indicates the proportion of variance in the dependent variable explained by the model. A value closer to 1 signifies a better fit.
4. **MAPE:** Expresses prediction accuracy as a percentage of the actual values, making it useful for comparing forecast performance across different datasets or scales.

Paper's Findings

Table 3. Evaluation metrics for forecasting residential sector energy consumption.

	CNN	BiLSTM	CNN_BiLSTM	CatBoost	LGBM	XGBoost
	lr = 0.1 k = 3	lr = 0.01 Hidden layer (h) = 50	lr = 0.01 (h) = 50 k = 3	lr = 0.01 Depth = 10	lr = 0.01 Depth = 5	lr = 0.5 Depth = 5
MSE	85400.36	6890.70	4570.14	9500.11	9100.86	9452.10
MAPE	20.29	5.44	4.98	7.92	7.35	7.45
RMSE	292.45	82.92	67.60	97.48	95.44	97.23
MAE	227.90	50.21	46.10	76.90	72.10	75.26
R^2	0.17	0.93	0.96	0.90	0.92	0.91

Paper’s Findings (cont.)



Phase 2 & 3 Roadmap

01

Standalone CNN

02

Standalone BiLSTM

03

CNN-BiLSTM

04

GB Family

05

Plotting

06

Analysis

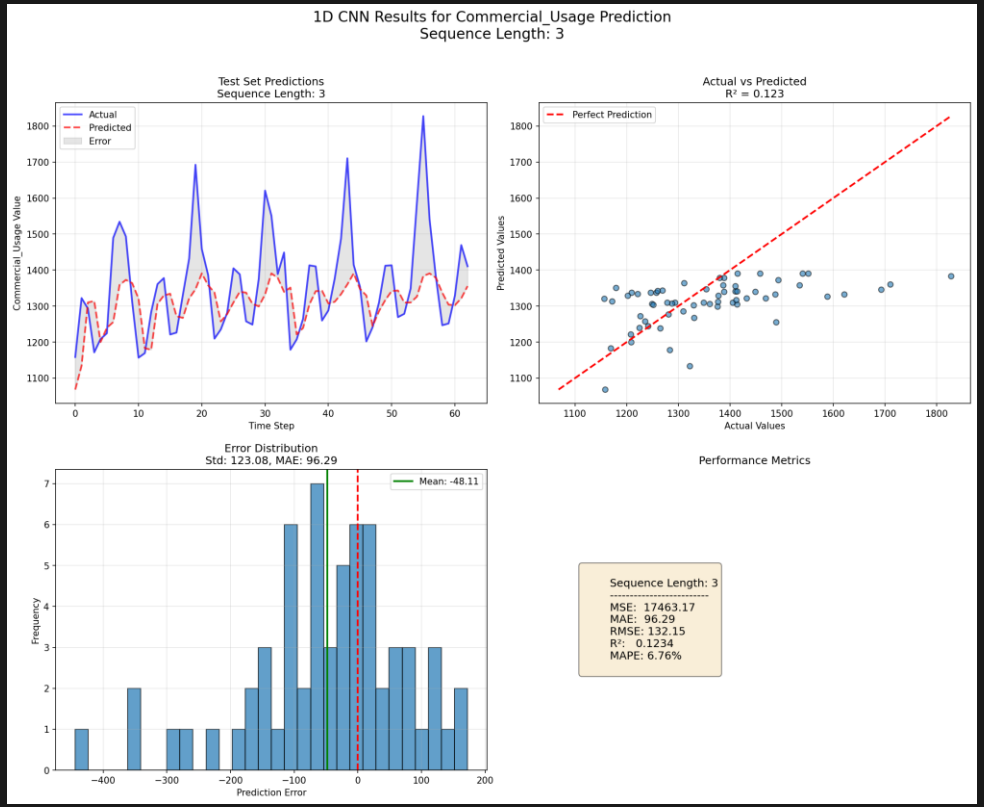
Standalone CNN: 1D-CNN

- **Paper's Approach:**
  1. 1D-CNN as spatial feature extractor
  2. Weight sharing for parameter reduction
  3. 1D max-pooling for feature compression
  4. Grid Search for hyperparameter optimization
- **My Implementation:**
  1. Three FC layers with Dropout
  2. Flexible sequence lengths (1, 3, 12)
  3. Full Grid Search with Early Stopping
  4. **Pipeline:** MinMaxScaler → sequence prep → PyTorch format

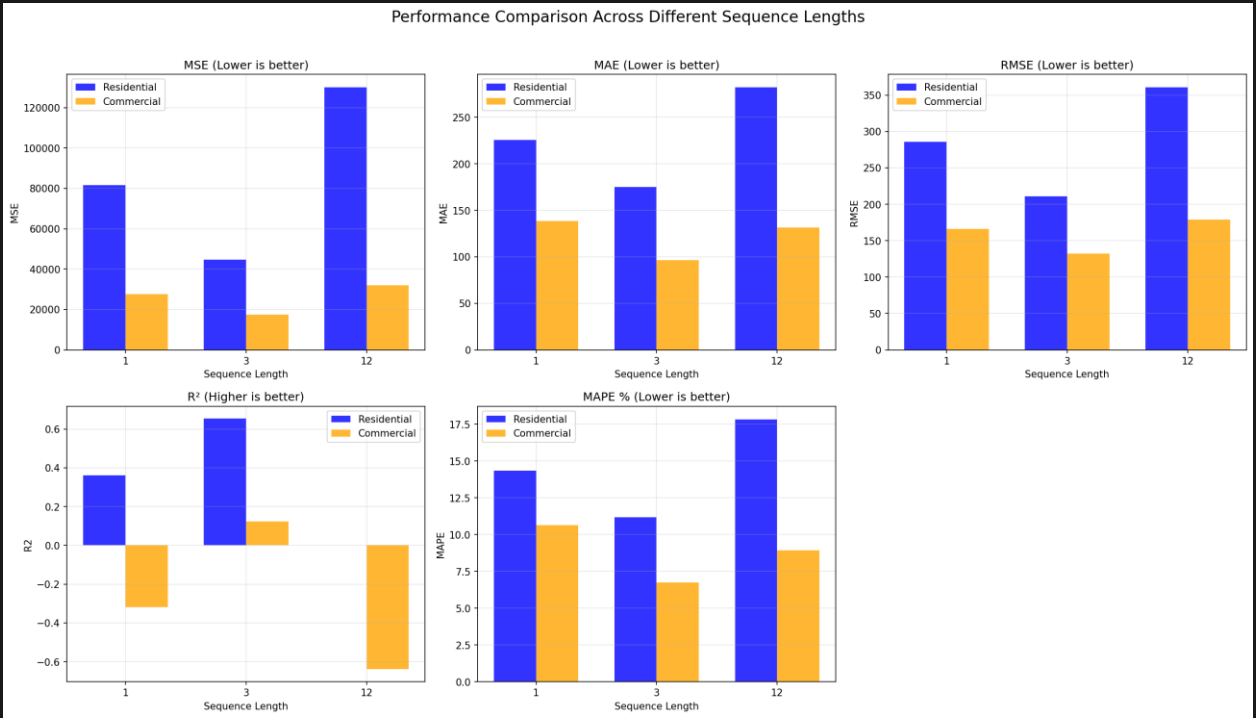
Standalone CNN: Implementation

- **Model Architecture (**Simple1DCNN **class):**
  - Two 1D convolutional layers with Batch Normalization and ReLU activation
  - Flattening layer followed by three fully connected layers with Dropout (configurable rate)
  - Final output layer with single neuron for regression
- **Key Hyperparameters (Grid Search):**
  - conv_filters: Number of filters in conv layers (tested: 1, 32, 64)
  - kernel_size: Convolution window size (tested: 3, 5)
  - fc1_units/fc2_units: Dense layer sizes (tested: [4,8] and [2,4] combinations)
  - dropout_rate: Regularization (tested: 0.05, 0.1)
  - learning_rate: Optimizer step size (tested: 0.05, 0.1)

Standalone CNN: 1D-CNN (cont.)



Standalone CNN: 1D-CNN (cont.)



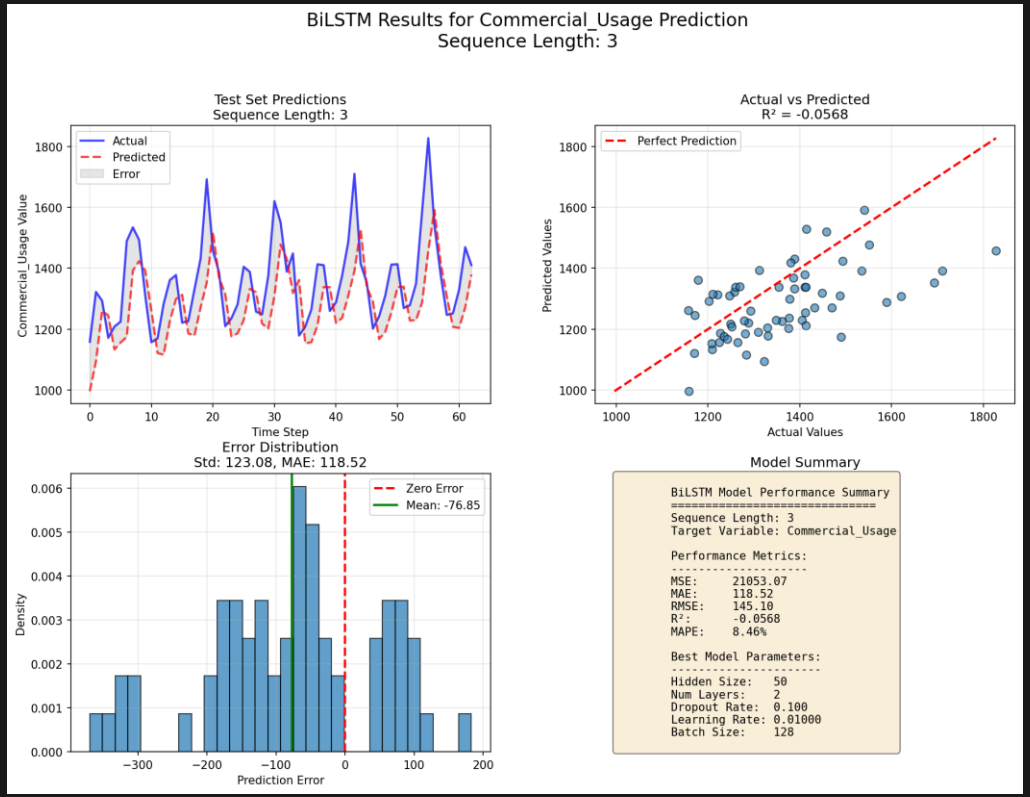
Standalone BiLSTM

- **Paper's Approach:**
  1. Bidirectional processing: forward + backward
  2. Grid Search for hyperparameter optimization
- **My Implementation:**
  1. 2-layer BiLSTM → concatenated hidden states
  2. Three FC layers with ReLU & Dropout
  3. Gradient Clipping for stability

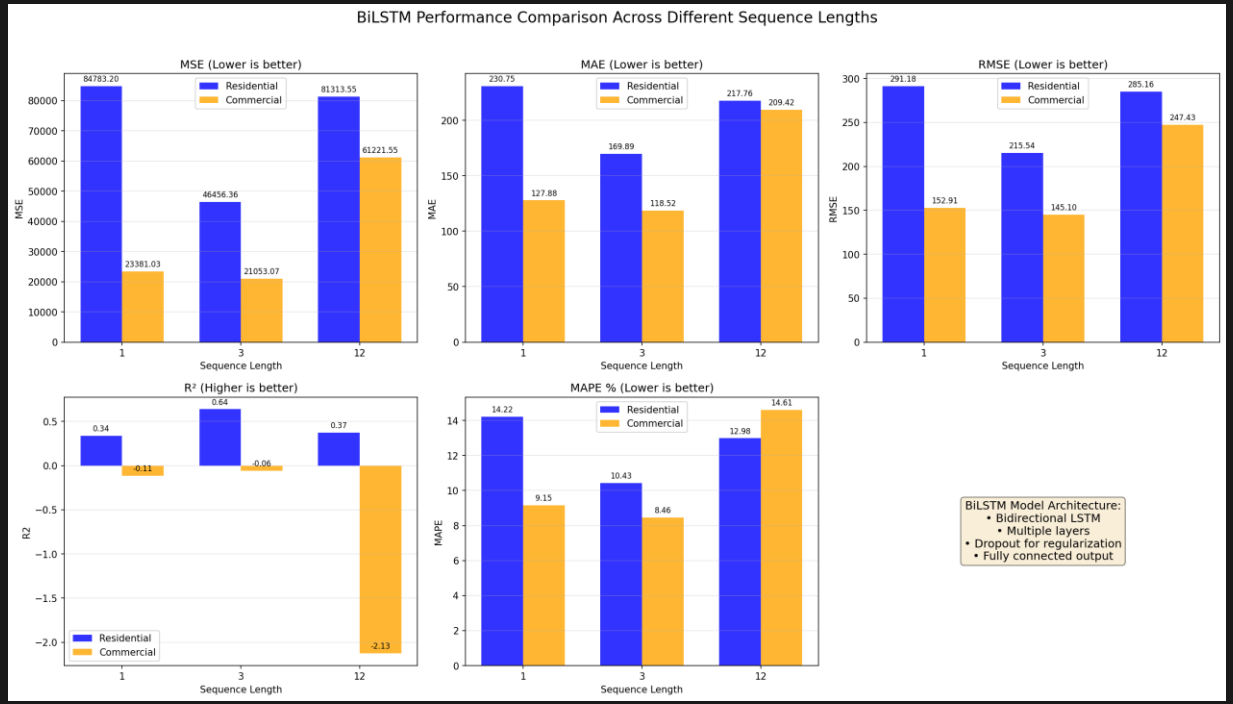
Standalone BiLSTM: Implementation

- **Model Architecture (**BiLSTMModel **class):**
  - Bidirectional LSTM layer (output size = hidden_size * 2)
  - Multiple stacked LSTM layers with dropout between layers
  - Three fc layers with ReLU activation and dropout for regression output
- **Key Hyperparameters (Grid Search):**
  - hidden_size: LSTM memory cells (tested: 32, 50, 64) - controls model capacity
  - num_layers: Stacked LSTM depth (tested: 2, 3) - deeper temporal abstraction
  - fc1_units/fc2_units: Dense layer sizes (tested: [8,16] and [4,8] combinations)
  - dropout_rate: Regularization (tested: 0.1, 0.2, 0.3) - prevents overfitting
  - learning_rate: Optimizer step size with ReduceLROnPlateau scheduler (factor=0.5, patience=5)

Standalone BiLSTM (cont.)



Standalone BiLSTM (cont.)



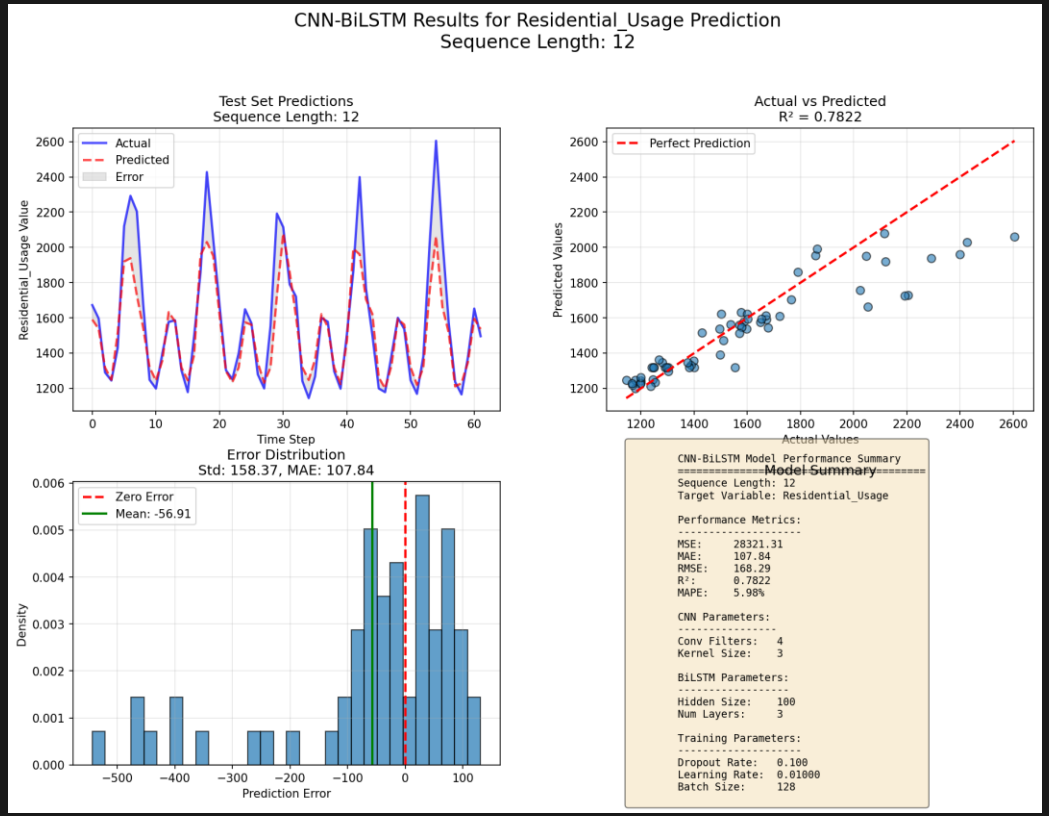
Proposed CNN-BiLSTM

- **Paper's Approach:**
  1. 1D-CNN: Spatial feature extraction
  2. BiLSTM: Temporal dependency learning
  3. Grid Search for optimal hyperparameters
- **My Implementation**
  1. **Deep CNN Backbone:** 3-layer Sequential CNN
     - Increasing filters (Filters → 2x → 4x)
     - BatchNorm + Dropout per layer

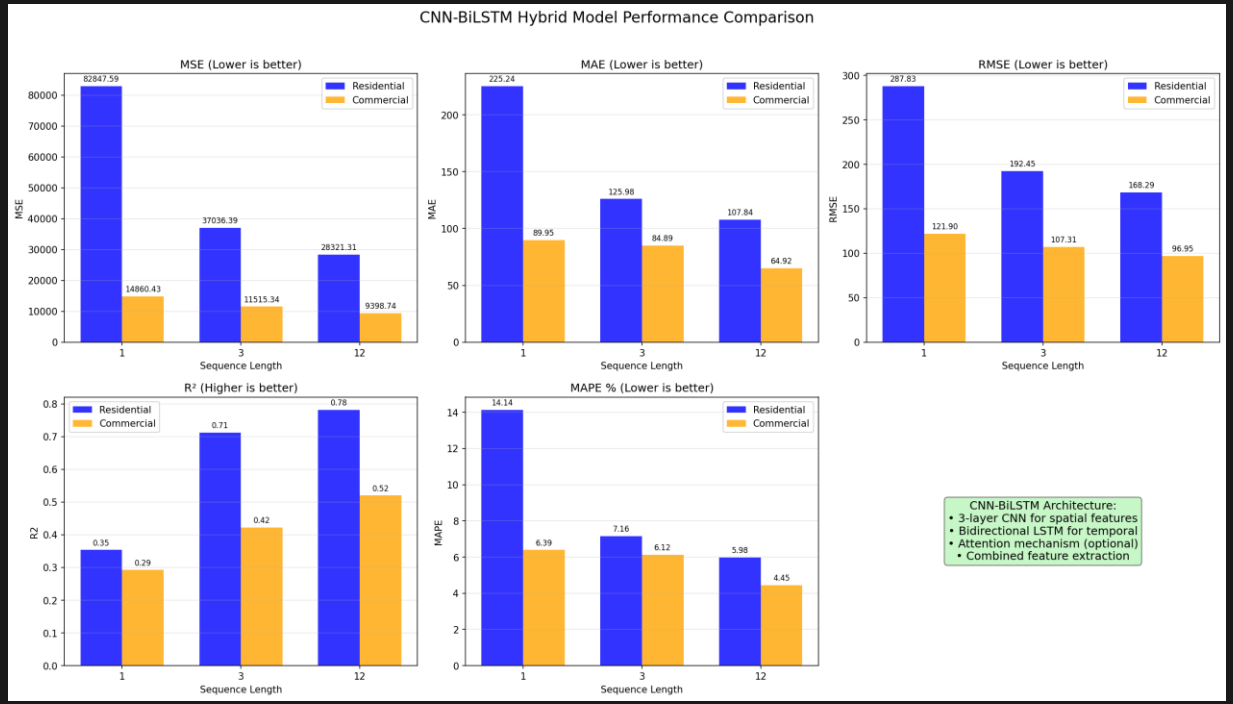
Proposed CNN-BiLSTM: Implementation

- **Model Architecture (**CNNBiLSTM **class):**
  - **3-layer CNN** with BatchNorm, ReLU, and Dropout for spatial feature extraction (conv_filters doubling each layer: 32→64→128)
  - **Optional attention layer** that computes weighted sum of LSTM outputs (vs. using last time step)
- **Key Hyperparameters (Grid Search):**
  - **CNN Section:** conv_filters (tested: 2, 4, 8, 16, 32) and kernel_size=3 for local pattern extraction
  - **LSTM Section:** lstm_hidden_size (tested: 50, 64, 100) and lstm_num_layers (tested: 2, 3, 4)
  - **Regularization:** dropout_rate=0.1, weight_decay=1e-5 (L2 regularization), gradient clipping (max_norm=1.0)
  - **Training:** learning_rate=0.01 with ReduceLROnPlateau scheduler, batch_size=128

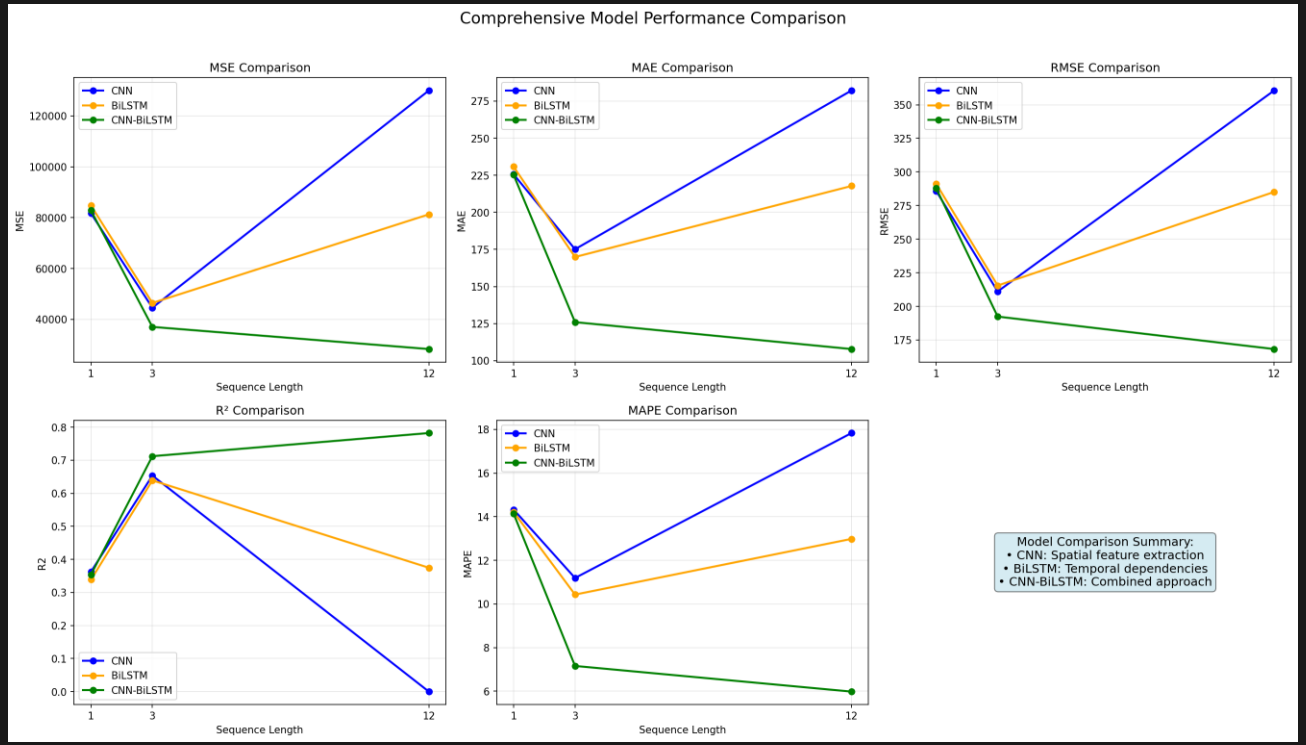
Proposed CNN-BiLSTM (cont.)



CNN-BiLSTM (cont.)



CNN-BiLSTM vs Standalone Models



Gradient Boosting Models (LGBM)

- **LightGBM:**
  - **Paper's Use:**
    1. Critical benchmark vs. CNN-BiLSTM
    2. Leaf-wise tree growth → lower loss
    3. Fast, efficient training baseline
  - **My Implementation:**
    1. Lag Feature Engineering
    2. Rolling Statistics & EWMA
    3. Cyclical Time Encoding (sine/cosine)

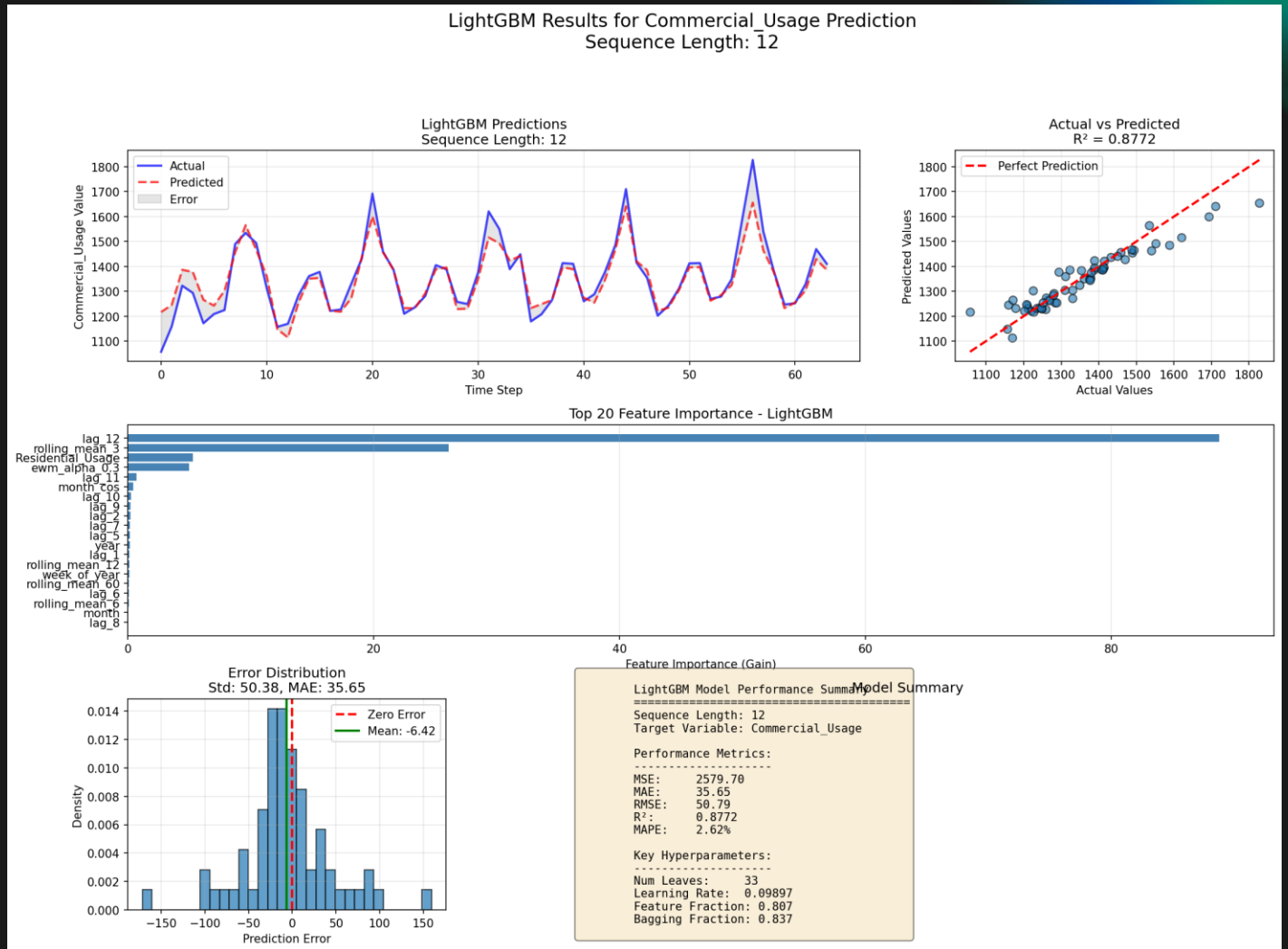
Gradient Boosting Models (XGBoost)

- **XGBoost:**
  - **Paper's Use:**
    1. State-of-the-art baseline
    2. Handles sparse IoT data well
    3. Proves deep learning superiority
  - **My Implementation**
    1. Feature Engineering Pipeline
    2. Regularization Tuning
    3. Optuna Sampler for optimization

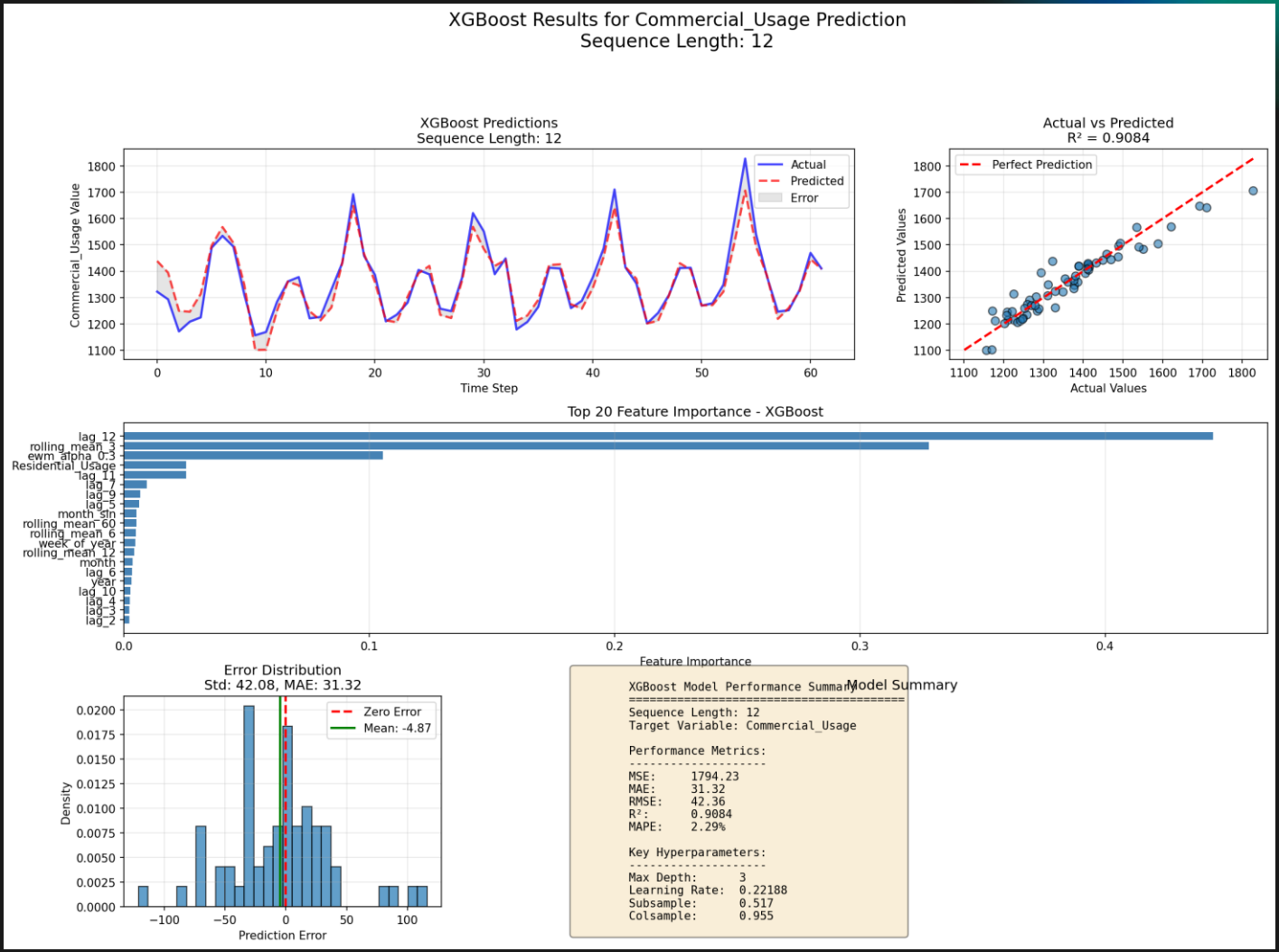
Gradient Boosting Models (CatBoost)

- **CatBoost:**
  - **Paper's Use:**
    1. "Ordered Boosting" for categorical features
    2. Symmetric trees as regularization
    3. Robust baseline for significance testing
  - **My Implementation:**
    1. Time-Series → Tabular transformation
    2. CatBoost-specific parameter tuning
    3. Feature Importance insights for lag analysis

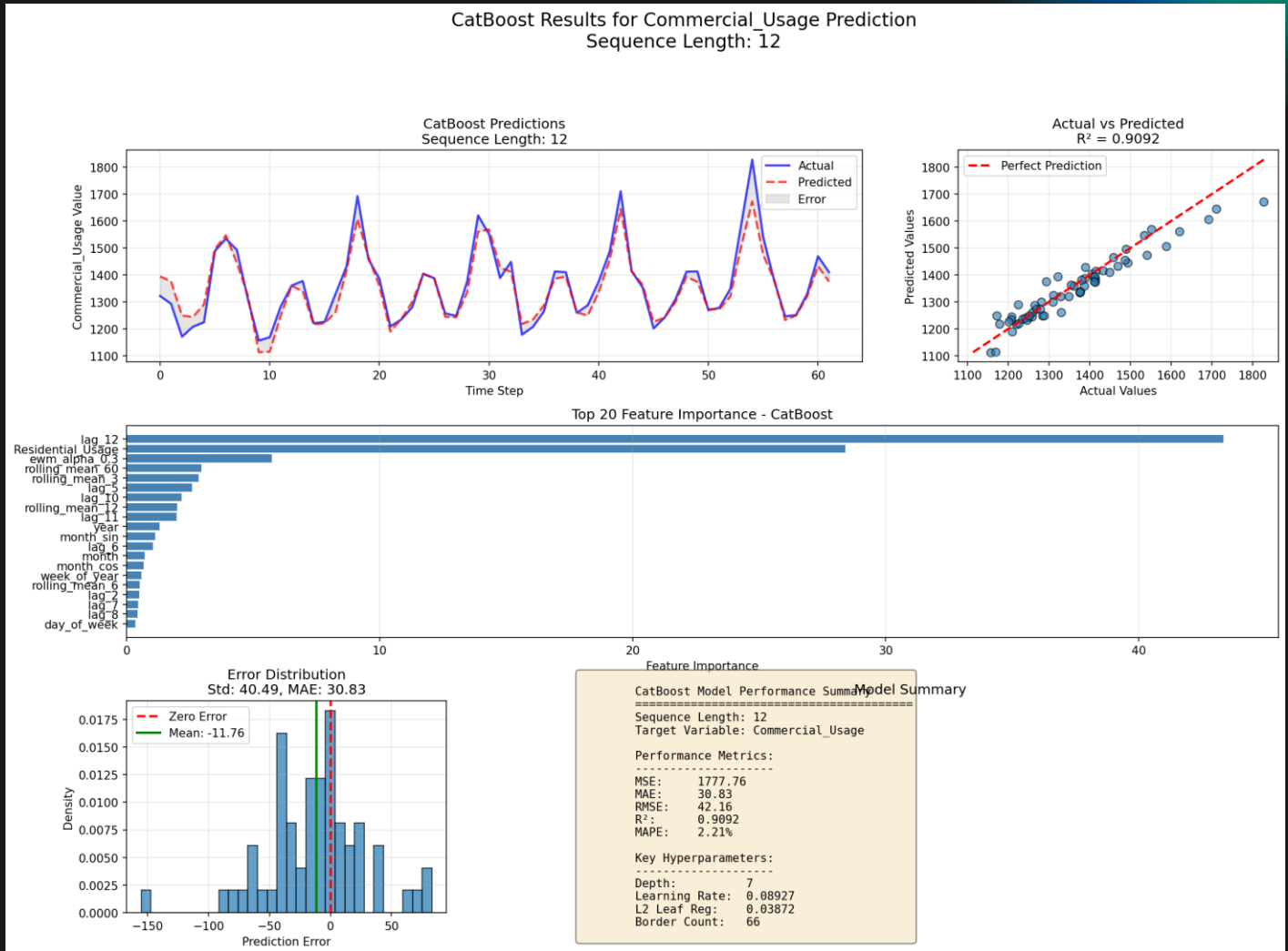
LGBM


LightGBM Results for Commercial_Usage Prediction
Sequence Length: 12
LightGBM Predictions
Sequence Length: 12
Actual
Predicted
Error
Commercial_Usage Value
Time Step
Actual vs Predicted
R² = 0.8772
Perfect Prediction
Predicted Values
Actual Values
Top 20 Feature Importance - LightGBM
lag_12
rolling_mean_3
Residential_Usage
ewm_alpha_0.3
lag_11
month_cos
lag_10
lag_9
lag_2
lag_7
lag_5
year
lag_1
rolling_mean_12
week_of_year
rolling_mean_60
lag_6
rolling_mean_6
month
lag_8
Feature Importance (Gain)
Error Distribution
Std: 50.38, MAE: 35.65
Zero Error
Mean: -6.42
Density
Prediction Error
Model Summary
LightGBM Model Performance Summary
Sequence Length: 12
Target Variable: Commercial_Usage
Performance Metrics:
MSE: 2579.70
MAE: 35.65
RMSE: 50.79
R²: 0.8772
MAPE: 2.62%
Key Hyperparameters:
Num Leaves: 33
Learning Rate: 0.09897
Feature Fraction: 0.807
Bagging Fraction: 0.837

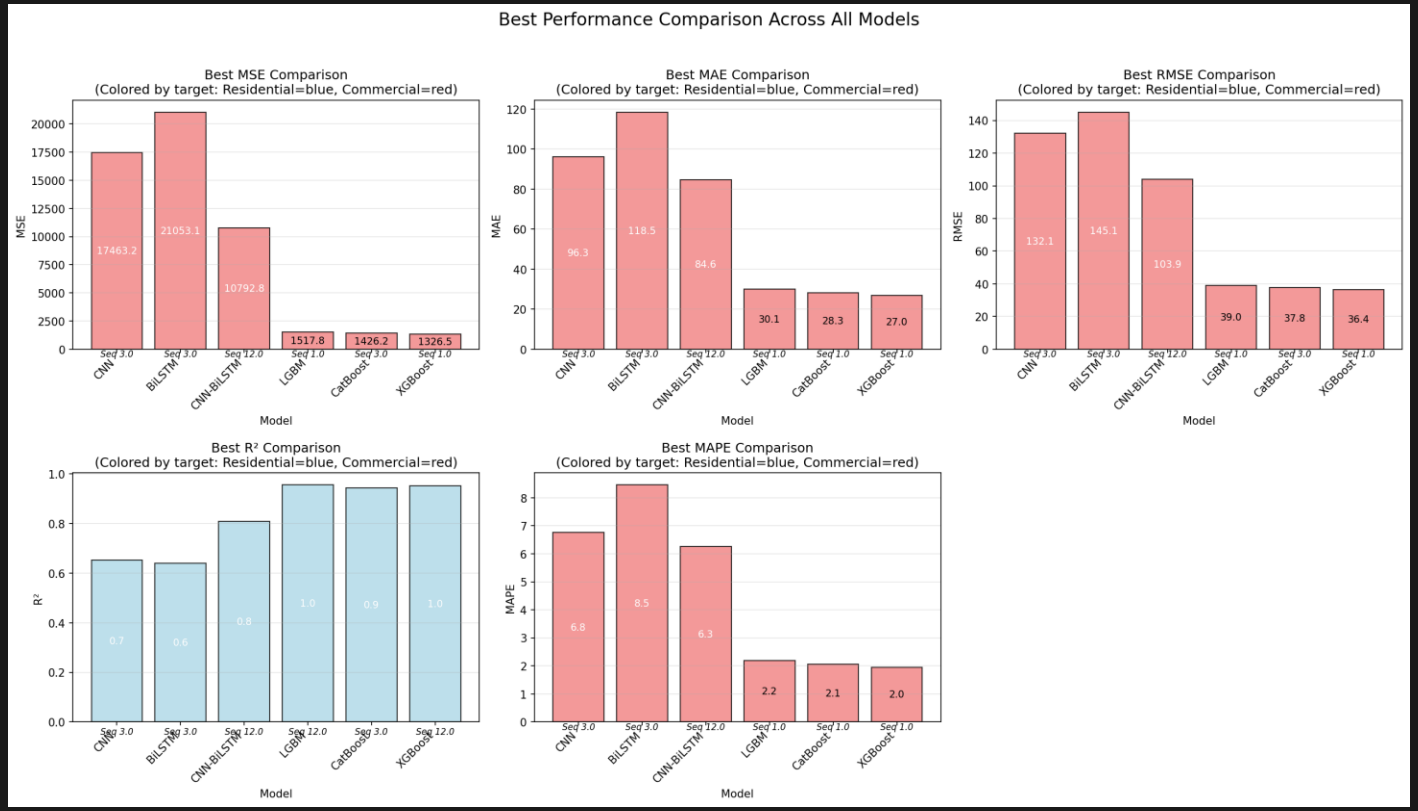
XGBoost


XGBoost Results for Commercial_Usage Prediction
Sequence Length: 12
XGBoost Predictions
Sequence Length: 12
Commercial_Usage Value
Time Step
Actual
Predicted
Error
Actual vs Predicted
R² = 0.9084
Perfect Prediction
Predicted Values
Actual Values
Top 20 Feature Importance - XGBoost
lag_12
rolling_mean_3
ewm_alpha_0.3
Residential_Usage
lag_11
lag_7
lag_9
lag_5
month_sin
rolling_mean_60
rolling_mean_6
week_of_year
rolling_mean_12
month
lag_6
year
lag_10
lag_4
lag_3
lag_2
Feature Importance
Error Distribution
Std: 42.08, MAE: 31.32
Zero Error
Mean: -4.87
Density
Prediction Error
Model Summary
XGBoost Model Performance Summary
Sequence Length: 12
Target Variable: Commercial_Usage
Performance Metrics:
MSE: 1794.23
MAE: 31.32
RMSE: 42.36
R²: 0.9084
MAPE: 2.29%
Key Hyperparameters:
Max Depth: 3
Learning Rate: 0.22188
Subsample: 0.517
Colsample: 0.955

CatBoost


CatBoost Results for Commercial_Usage Prediction
Sequence Length: 12
CatBoost Predictions
Sequence Length: 12
Commercial_Usage Value
Time Step
Actual
Predicted
Error
Actual vs Predicted
R² = 0.9092
Perfect Prediction
Predicted Values
Actual Values
Top 20 Feature Importance - CatBoost
lag_12
Residential_Usage
ewm_alpha_0.3
rolling_mean_60
rolling_mean_3
lag_5
lag_10
rolling_mean_12
lag_11
year
month_sin
lag_6
month
month_cos
week_of_year
rolling_mean_6
lag_2
lag_7
lag_8
day_of_week
Feature Importance
Error Distribution
Std: 40.49, MAE: 30.83
Zero Error
Mean: -11.76
Density
Prediction Error
Model Summary
CatBoost Model Performance Summary
Sequence Length: 12
Target Variable: Commercial_Usage
Performance Metrics:
MSE: 1777.76
MAE: 30.83
RMSE: 42.16
R²: 0.9092
MAPE: 2.21%
Key Hyperparameters:
Depth: 7
Learning Rate: 0.08927
L2 Leaf Reg: 0.03872
Border Count: 66

Performance Comparison



Performance Table

Best Performance Summary Across All Models

Model	MSE	MAE	RMSE	R^2	MAPE
CNN	17463.17	96.29	132.15	0.65	6.76
BiLSTM	21053.07	118.52	145.10	0.64	8.46
CNN-BiLSTM	10792.80	84.64	103.89	0.81	6.26
LGBM	1517.77	30.05	38.96	0.96	2.20
CatBoost	1426.18	28.29	37.76	0.94	2.06
XGBoost	1326.47	27.03	36.42	0.95	1.95

Key Questions

1. **Our GB implementations consistently outperform published results across all metrics. But why?**
    - GB models are exceptionally efficient at handling tabular data with clear "if-then" logic. Because I implemented sophisticated **Feature Engineering** (Lags and Rolling Statistics), I essentially provided the GB models with the "answers" that the CNN was trying to learn from scratch. In datasets of this size, explicit feature engineering often rivals or beats automatic feature extraction.

2. **Why is the integration of IoT data critical for modern building energy management systems?**
    - Traditional energy models often rely on aggregate utility data, which lacks granularity. IoT sensors provide high-frequency, real-time data at the device or floor level, moving from general estimation to individualized prediction.

Key Questions (cont.)

3. **What is the main takeaway regarding model selection for energy forecasting?**
    - While the CNN-BiLSTM is superior to basic NNs, best model depends on the available computational resources and feature engineering. if you are dealing with raw, unstructured IoT streams, the **Proposed Hybrid CNN-BiLSTM** is the most robust path forward for automated, scalable forecasting.

4. **Why did the Hybrid CNN-BiLSTM scale better with longer sequences (12 months) compared to standalone models?**
    - Standalone LSTMs often suffer from catastrophic forgetting or signal noise when the input window is too large. In my results, the CNN layers effectively compressed the 12-month data into meaningful feature maps. This simplified the BiLSTM's job, allowing it to capture annual seasonality without being overwhelmed by 365 days of raw, noisy data points.

Key Questions (cont.)

5. **What do we conclude regarding the relationship between "prediction accuracy" and "global sustainability goals"?**

    - The paper argues that accuracy is not just a technical metric but a sustainability tool. Accurate forecasting allows for the integration of renewable energy sources (which are intermittent) into the grid. By knowing exactly how much energy a building will need, we can reduce the reliance on "peaker" power plants (which are often high-emission) and minimize energy waste, directly contributing to carbon reduction.

Thanks!

Any Questions ?

CREDITS: This presentation template was created by **Slidesgo**, and includes icons, infographics & images by **Freepik**